

B.TECH ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Algorithm Real-Life Application Assignment

Algorithms Covered: Breadth-First Search (BFS) and A (A-Star) Search*

Field	Details
Name	Syeed Hasan SH
Roll Number	25AD170
Course	B.Tech Artificial Intelligence and Data Science
Subject	25AD301- AI
Assignment Title	Algorithm Real-Life Application (BFS & A*)

Experiment 1: Breadth-First Search (BFS)

1. Experiment Number

Experiment No. 1

2. Title

Breadth-First Search (BFS) — Finding Minimum Connections in a Social Network

3. Aim

To implement the Breadth-First Search (BFS) algorithm using Python and demonstrate its real-life application in finding the shortest path (minimum number of connections) between two people in a social network.

4. Introduction / Short Description

Breadth-First Search (BFS) is a fundamental graph traversal algorithm that explores a graph level by level, starting from a source node. It uses a queue (First-In-First-Out data structure) to keep track of nodes that need to be visited. BFS visits all neighbours of a node before moving to the next level, which guarantees that the first time it reaches the goal node, it has done so using the minimum number of edges — making it ideal for finding shortest paths in unweighted graphs.

5. Algorithm

1. Create an empty queue and add the start node to it. Mark the start node as visited.
2. While the queue is not empty, remove (dequeue) the node at the front of the queue — call it the current node.
3. If the current node is the goal node, stop the search.
4. Otherwise, for every neighbour of the current node that has not been visited: mark it as visited, record its parent (the current node), and add it to the back of the queue.
5. Repeat steps 2–4 until the goal is found or the queue becomes empty.
6. Reconstruct the shortest path by following parent pointers backward from the goal node to the start node, then reverse the path.

6. Real-Life Application

Social networking platforms such as LinkedIn and Facebook use BFS-based logic to compute 'degrees of separation' — for example, showing that a user is a '2nd degree connection' or '3rd degree connection' of another user. BFS is also used in network broadcasting, peer-to-peer file sharing, web crawling, and finding the shortest route on an unweighted road/network map.

7. Explanation of How the Algorithm Is Used in That Real-Life Application

In a social network, each person is a node and each friendship/connection is an edge. When a platform needs to show how closely two users are connected, it runs BFS starting from the source user. BFS explores all direct friends first (1 hop away), then friends-of-friends (2 hops away), and so on, level by level. Because BFS always finds nodes in increasing order of distance from the source, the very first time it reaches the target user, that path is guaranteed to use the minimum number of connections. This is exactly why BFS — and not a random traversal like DFS — is used for this feature.

8. Problem Statement

Given a small social network represented as a graph (people as nodes, friendships as edges), find the minimum number of connections (shortest path) required to connect two given people, Alice and Henry, and display the exact chain of connections.

9. Input

Adjacency list representing the social network (8 people and their direct friendships), a start person 'Alice', and a goal person 'Henry'.

10. Expected Output

- Order in which people are visited (BFS traversal order)
- Queue behaviour at each step
- The shortest chain of connections from Alice to Henry
- The minimum number of connections (steps/hops)

11. Python Code

```
# =====
# Experiment 1: Breadth-First Search (BFS)
# Real-Life Application: Finding the Minimum Number of
# Connections (Degrees of Separation) in a Social Network
# =====

from collections import deque

# -----
# Step 1: Represent the social network as an adjacency list
# Each person (node) is connected to their direct friends
# -----
social_network = {
    'Alice': ['Bob', 'Carol'],
    'Bob':   ['Alice', 'David', 'Eve'],
    'Carol': ['Alice', 'Frank'],
    'David': ['Bob', 'Grace'],
    'Eve':   ['Bob', 'Grace'],
    'Frank': ['Carol', 'Grace'],
    'Grace': ['David', 'Eve', 'Frank', 'Henry'],
    'Henry': ['Grace']
}

def bfs_shortest_path(graph, start, goal):
    """
    Performs BFS starting from 'start' node to find the
    shortest path (minimum number of connections) to 'goal'.
    Returns the path as a list and prints the traversal steps.
    """

    # visited keeps track of nodes we have already added to the queue
    visited = set([start])

    # queue holds the frontier nodes to explore (FIFO order)
    queue = deque([start])

    # parent dictionary is used to reconstruct the shortest path
    parent = {start: None}

    step = 0
    print("Starting BFS Traversal")
    print("-" * 50)

    # -----
    # Step 2: Standard BFS loop using a queue
    # -----
    while queue:
        step += 1
        current = queue.popleft()          # remove the front person from the queue
        print(f"Step {step}: Visiting '{current}' | Queue before pop: {list(queue)}")

        # Goal check: stop as soon as we reach the target person
```

```

        if current == goal:
            print(f"\nGoal '{goal}' found!")
            break

        # Explore all direct friends (neighbours) of the current person
        for neighbour in graph[current]:
            if neighbour not in visited:
                visited.add(neighbour)          # mark as seen so we don't add it twice
                parent[neighbour] = current      # remember how we reached this neighbour
                queue.append(neighbour)          # add neighbour to the back of the queue
                print(f"        -> Added '{neighbour}' to queue")

    # -----
    # Step 3: Reconstruct the shortest path using parent pointers
    # -----
    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent.get(node)
    path.reverse()

    return path

# -----
# Step 4: Define the start person and the goal person
# -----
start_person = 'Alice'
goal_person = 'Henry'

print(f"Social Network Graph: {social_network}\n")
print(f"Finding minimum connections between '{start_person}' and '{goal_person}'\n")

shortest_path = bfs_shortest_path(social_network, start_person, goal_person)

# -----
# Step 5: Display the final result
# -----
print("-" * 50)
print("RESULT")
print("-" * 50)
print("Shortest Path :", " -> ".join(shortest_path))
print("Minimum Number of Connections (Steps) :", len(shortest_path) - 1)

```

Output Link: <https://tinyurl.com/4t2my3jw>

12. Step-by-Step Explanation of the Code

1. The social network is defined as a Python dictionary (adjacency list) where each key is a person and the value is a list of their direct friends.
2. The function `bfs_shortest_path()` initializes a visited set (containing only the start node), a queue (`collections.deque`) containing the start node, and a parent dictionary to later rebuild the path.
3. Inside the while loop, the algorithm removes the front person from the queue using `popleft()` — this is the FIFO behaviour that defines BFS.
4. If the removed person is the goal, the loop breaks immediately since the shortest path has been found.
5. Otherwise, every unvisited neighbour of the current person is marked visited, its parent is recorded, and it is appended to the back of the queue using `append()`.
6. After the loop ends, the code walks backward from the goal node using the parent dictionary to rebuild the path, then reverses it to get the path from start to goal.

7. Finally, the program prints the shortest path and the number of connections (path length – 1).

16. Result

The Breadth-First Search algorithm was successfully implemented in Python. The minimum number of connections between 'Alice' and 'Henry' in the given social network was found to be 4, via the path Alice → Bob → David → Grace → Henry.

17. Time Complexity

$O(V + E)$, where V is the number of nodes (people) and E is the number of edges (friendships), since every node and every edge is examined at most once.

18. Space Complexity

$O(V)$, for storing the visited set, the queue, and the parent dictionary, each of which can hold up to V nodes in the worst case.

Experiment 2: A* (A-Star) Search Algorithm

1. Experiment Number

Experiment No. 2

2. Title

A* (A-Star) Search Algorithm — GPS/Road Route Finding Between Cities

3. Aim

To implement the A* search algorithm using Python and demonstrate its real-life application in finding the optimal (least-cost) route between two cities on a weighted road network.

4. Introduction / Short Description

A* (pronounced 'A-star') is an informed search algorithm that finds the least-cost path between a start node and a goal node in a weighted graph. It combines the actual cost travelled so far, $g(n)$, with a heuristic estimate of the remaining cost to the goal, $h(n)$, to compute $f(n) = g(n) + h(n)$ for every node. A* always expands the node with the lowest $f(n)$ value next, which makes it more efficient than uninformed algorithms while still finding an optimal path — provided the heuristic used is admissible (never overestimates the true remaining cost).

5. Algorithm

1. Initialize an open list containing only the start node, with $g(\text{start}) = 0$.
2. While the open list is not empty, select the node with the smallest $f(n) = g(n) + h(n)$ value — call it the current node.
3. If the current node is the goal node, stop; the optimal path has been found.
4. Otherwise, move the current node to the closed list (fully processed nodes).
5. For each neighbour of the current node: compute the new g value (g of current + edge cost). If this neighbour is new, or the new g value is smaller than any previously recorded g value for it, update its g value, set its parent to the current node, and add/update it in the open list.
6. Repeat steps 2–5 until the goal is reached or the open list becomes empty (no path exists).
7. Reconstruct the optimal path by following parent pointers backward from the goal to the start, then reverse it.

6. Real-Life Application

GPS navigation systems (such as Google Maps) and robot/vehicle path-planning systems use A*-style algorithms to compute the fastest or shortest route between a source location and a destination on a road network or grid map, while avoiding an exhaustive search of every possible route.

7. Explanation of How the Algorithm Is Used in That Real-Life Application

In GPS navigation, cities/junctions are nodes and roads are weighted edges (weights represent distance or time). The actual road distance already travelled from the start corresponds to $g(n)$. The heuristic $h(n)$ is typically the straight-line ('as the crow flies') distance from a location to the destination, which is quick to compute and never overestimates the true remaining road distance. By always expanding the location with the lowest estimated total cost $f(n)$, A* avoids exploring routes that are clearly worse, letting GPS systems compute optimal routes over huge road networks in real time instead of checking every possible path.

8. Problem Statement

Given a small weighted road network of six cities with realistic road distances and an admissible heuristic (straight-line distance estimate to the destination), find the optimal (minimum-cost) route from Chennai to Bangalore using A* search.

9. Input

- A weighted adjacency dictionary (road_network) representing 6 cities and the road distance between connected cities.
- A heuristic dictionary (heuristic) giving the estimated straight-line distance from each city to the destination, Bangalore.
- Start city: Chennai. Goal city: Bangalore.

10. Expected Output

- The sequence of cities expanded by A*, along with their g(n), h(n), and f(n) values at each step
- The final optimal route from Chennai to Bangalore
- The total cost (distance) of that optimal route

11. Python Code

```
# =====
# Experiment 2: A* (A-Star) Search Algorithm
# Real-Life Application: GPS Navigation / Road Route Finding
# =====

# -----
# Step 1: Represent the road network as a weighted graph
# Numbers on each edge represent the road distance (in km)
# between two cities.
# -----
road_network = {
    'Chennai':      {'Vellore': 4, 'Kanchipuram': 2},
    'Vellore':      {'Chennai': 4, 'Kanchipuram': 1, 'Salem': 5},
    'Kanchipuram':  {'Chennai': 2, 'Vellore': 1, 'Krishnagiri': 7},
    'Salem':         {'Vellore': 5, 'Krishnagiri': 3, 'Bangalore': 6},
    'Krishnagiri':   {'Kanchipuram': 7, 'Salem': 3, 'Bangalore': 2},
    'Bangalore':     {'Salem': 6, 'Krishnagiri': 2}
}

# -----
# Step 2: Heuristic values h(n) = estimated straight-line
# distance (in km) from each city to the destination (Bangalore)
# NOTE: These values are chosen so that they never OVERESTIMATE
# the true remaining distance -> this makes the heuristic
# "admissible", which is required for A* to guarantee an
# optimal (shortest) path.
# -----
heuristic = {
    'Chennai':      7,
    'Vellore':      6,
    'Kanchipuram':  6,
    'Salem':         3,
    'Krishnagiri':   1,
    'Bangalore':     0
}

def a_star_search(graph, heuristic, start, goal):
    """
    Performs A* search from 'start' to 'goal'.

    f(n) = g(n) + h(n)
    g(n) -> actual cost travelled so far from start to node n
    h(n) -> estimated (heuristic) cost from node n to goal
    f(n) -> estimated total cost of the cheapest path through n

    A* always expands the node with the lowest f(n) value next.
    Returns the optimal path and its total cost.
    """
```

```

# open_list stores nodes yet to be explored: each entry is (node, g_cost)
open_list = [(start, 0)]

# g_cost keeps the best known cost from start to each node
g_cost = {start: 0}

# parent is used to reconstruct the final path
parent = {start: None}

# closed_set stores nodes that are already fully processed
closed_set = set()

step = 0
print("Starting A* Search")
print("-" * 70)

while open_list:
    step += 1

    # -----
    # Step 3: Pick the node in open_list with smallest f(n)
    # (simple linear search - easy to visualise in PyTutor)
    # -----
    best_index = 0
    best_node, best_g = open_list[0]
    best_f = best_g + heuristic[best_node]
    for i in range(1, len(open_list)):
        node, g_val = open_list[i]
        f_value = g_val + heuristic[node]
        if f_value < best_f:
            best_f = f_value
            best_index = i

    current, current_g = open_list.pop(best_index)

    # Skip this entry if the node was already finalised earlier
    # through a cheaper path (this can happen because we use a
    # simple list instead of a priority queue with decrease-key)
    if current in closed_set:
        step -= 1
        continue

    print(f"Step {step}: Expanding '{current}' "
          f"| g={current_g} h={heuristic[current]} f={current_g + "
          heuristic[current]}")

    # Goal check
    if current == goal:
        print(f"\nGoal '{goal}' reached!")
        break

    closed_set.add(current)

    # -----
    # Step 4: Explore neighbours and update costs if a
    # cheaper path is found
    # -----
    for neighbour, edge_cost in graph[current].items():
        if neighbour in closed_set:
            continue

        new_g = current_g + edge_cost

        # If neighbour is new OR we found a cheaper path to it
        if neighbour not in g_cost or new_g < g_cost[neighbour]:

```



```

        g_cost[neighbour] = new_g
        parent[neighbour] = current
        open_list.append((neighbour, new_g))
        print(f"          -> Updated '{neighbour}': "
              f"g={new_g}  h={heuristic[neighbour]}  f={new_g + "
heuristic[neighbour]}")

# -----
# Step 5: Reconstruct the optimal path using parent pointers
# -----
path = []
node = goal
while node is not None:
    path.append(node)
    node = parent[node]
path.reverse()

return path, g_cost[goal]

# -----
# Step 6: Define start city and destination city
# -----
start_city = 'Chennai'
goal_city = 'Bangalore'

print(f"Road Network: {road_network}\n")
print(f"Finding optimal route from '{start_city}' to '{goal_city}'\n")

optimal_path, total_cost = a_star_search(road_network, heuristic, start_city,
goal_city)

# -----
# Step 7: Display the final result
# -----
print("-" * 70)
print("RESULT")
print("-" * 70)
print("Optimal Route      :", " ".join(optimal_path))
print("Total Route Cost   :", total_cost, "km")

```

Output Link: <https://tinyurl.com/53s4m4x4>

12. Step-by-Step Explanation of the Code

1. The road network is defined as a dictionary of dictionaries: each city maps to its directly connected neighbouring cities and the road distance to each.
2. The heuristic dictionary stores an admissible estimated distance from every city to the destination, Bangalore — these values never exceed the true shortest remaining distance.
3. The function `a_star_search()` maintains an `open_list` (list of (node, `g_cost`) tuples still to be explored), a `g_cost` dictionary (best known cost from start to each node), a parent dictionary (for path reconstruction), and a `closed_set` (nodes fully processed).
4. In every iteration, the code scans `open_list` to find the entry with the smallest $f(n) = g(n) + h(n)$ using a simple linear search — deliberately kept simple (instead of a heap) so it is easy to trace in PyTutor.
5. If the popped entry refers to a node already in `closed_set` (a stale, previously superseded entry), it is skipped — this is a simple 'lazy deletion' technique.
6. If the current node is the goal, the loop stops immediately.
7. Otherwise, every neighbour is examined: if a cheaper path to that neighbour has just been found, its `g_cost`, `parent`, and `open_list` entry are updated.

- After the loop, the optimal path is rebuilt using the parent dictionary and reversed, and the program prints the route and its total cost.

16. Result

The A* search algorithm was successfully implemented in Python. The optimal route from Chennai to Bangalore was found to be Chennai → Kanchipuram → Krishnagiri → Bangalore, with a total cost of 11 km, using an admissible heuristic that guaranteed optimality.

17. Time Complexity

In the worst case, A* can behave like $O(b^d)$, where b is the branching factor and d is the depth of the optimal solution. In practice, its actual performance depends heavily on how good (close to the true cost) the heuristic is — a better heuristic explores far fewer nodes.

18. Space Complexity

$O(V)$, since A* keeps every generated node in either the open list or the closed set in the worst case.

Comparison of BFS and A*

Criteria	BFS	A*
Type	Uninformed (blind) search	Informed (heuristic-based) search
Data Structure Used	Queue (FIFO)	Priority list/queue ordered by $f(n)$
Weighted Graph Support	No (assumes all edges equal cost)	Yes (uses actual edge weights)
Uses a Heuristic?	No	Yes — $h(n)$, estimated cost to goal
Typical Use	Shortest path in unweighted graphs, degrees of separation	Optimal route finding in weighted graphs, GPS/robot navigation
Time Complexity	$O(V + E)$	$O(b^d)$ worst case; depends on heuristic quality
Space Complexity	$O(V)$	$O(V)$
Real-Life Application	Social network minimum connections	GPS navigation / road route finding

Conclusion

This assignment demonstrated two important search algorithms used in Artificial Intelligence: Breadth-First Search (BFS), an uninformed algorithm suited to unweighted graphs such as finding minimum connections in a social network, and A* Search, an informed algorithm that combines actual cost and heuristic estimates to find optimal routes in weighted graphs such as GPS navigation. Both algorithms were implemented in Python, verified to run correctly, visualized step-by-step in PyTutor, and connected to clear, realistic real-life applications. Studying both algorithms side by side highlights how the choice between uninformed and informed search depends on whether edge weights and a reliable heuristic are available.